预演（Rehearsal）技术文档

传智杯 AI 创新应用挑战赛·赛道 id=10039

Contents

1. 问题与用户 **2**
- 1.1 真实场景 . . . 2
- 1.2 目标用户与使用时机 . . . 2
- 1.3 产品形态 . . . 2
- 1.4 明确的边界（不做什么） . . . 3

2. 意图控制策略与可证伪生成 **3**
- 2.1 五阶段 Prompt 链与各自的意图 . . . 3
- 2.2 上下文设计的四条硬规则 . . . 3
- 2.3 强制打分为什么会被打歪——以及怎么治 . . . 4
- 2.4 材料风险分为什么不由模型给 . . . 5
- 2.5 Prompt 演进证据：v1 vs v2 同输入对比 . . . 5
- 2.6 追问收敛：纯函数，不交给模型自觉 . . . 6
- 2.7 五阶段之外的守门：三道校验 . . . 6

3. 架构设计 **7**
- 3.1 技术栈与选型理由 . . . 7
- 3.2 分层与依赖方向 . . . 7
- 3.3 约束 AI 生成代码的六条机制 . . . 8
- 3.4 锚点模型：全篇最核心的数据结构 . . . 8
- 3.5 安全基线 S1–S6 . . . 9
- 3.6 SSE 流式与思考态 . . . 9
- 3.7 MOCK 回放：架构上的一等公民 . . . 9

4. 可复现性论证 **10**
- 4.1 三层可复现，从强到弱 . . . 10
- 4.2 一个命令 . . . 10
- 4.3 fresh clone 为什么曾经走不通（一个真实的可复现性故障） . . . 10
- 4.4 固定随机性的手段（以及为什么不够） . . . 11
- 4.5 锚点可复检 . . . 11
- 4.6 模型版本漂移的风险 . . . 11

5. 评测与数据 **11**
- 5.1 八项机检指标（`scripts/eval.mjs` 自动产出） . . . 12
- 5.2 双采样实测：哪些值恒定、哪些会变 . . . 12
- 5.3 三份真实使用记录 . . . 13
- 5.4 报告的"评分散点"：为什么按评委聚合 . . . 13
- 5.5 这些数字不能用来声称什么 . . . 14

6. 局限与下一步 **14**
- 6.1 已知局限（按严重程度排序） . . . 14
- 6.2 下一步（如果继续做） . . . 14

6.3 不打算做的 14
附录 A：AI 协同记录索引 15
附录 B：目录与文件索引 15

传智杯「AI 创新应用挑战赛」参赛作品·赛道 id=10039 ·个人赛 文档版本 v1.0 ｜ 2026-11-02 ｜对应代码 tag：`v1.0-freeze`

本文档写的是方法，不是功能清单。官方要求重点阐述三件事：你怎么设计上下文和 Prompt（第 2 章）、AI 生成的代码采用什么架构、你怎么约束它（第 3 章）、你怎么证明可复现（第 4 章）。第 1 章交代问题与用户，第 5 章给可机检的评测数字，第 6 章写局限。

---

1. 问题与用户

1.1 真实场景

学生、研究生、项目负责人在答辩前普遍面临同一个困境：**正式答辩只有一次机会，而”被质疑”这件事在一次机会里无法练习。**

已有的工具切不进这个场景：

已有做法	断在哪
找个通用 AI 聊天框问几句	问一次就结束，不会追到你答不上来；模型说”你缺数据支撑”，你翻半小时找不到是哪一句
学长/同学帮忙模拟答辩	人力有限、不可复现、不好意思连着追问十次
逐字背稿	背的是答案不是应答逻辑，一被追问就回到原稿

要练的不是”把答案说出来”，是”在被追问到第二层时还站得住”。这需要三件东西：连续性提问（会追）、可归因的结论（说得出凭什么）、可比较的结果（这次比上次好在哪）。

1.2 目标用户与使用时机

用户	时机	关心什么
本科/研究生毕业生	毕设答辩前 1–2 周	论文里哪些地方一问就露怯
项目答辩人（大创、竞赛）	提交材料后、正式答辩前	项目文档的论证漏洞在哪
求职者	面试前	简历上哪些经历经不起深挖

共同点：**材料是既有的、结构化的、篇幅有限的**（论文 / 项目文档 / 简历），且**风险可以提前识别。**

1.3 产品形态

上传材料 → 三位不同视角的 AI 评委轮番提问并追问 → 输出答辩风险报告，**每条结论都能一键追回原文那一句。**

两位评委是错的，一位更错：

- 一位评委 = 一个人的偏见，问完你不知道是真有问题还是他爱挑刺；
- 三位不同视角（方法学 / 工程 / 价值）= 分歧本身成为信息——两位说有问题、一位说没有，和三位都说有问题，含义完全不同。

这也是报告页为什么要画”评委打分散点”（§5.3）：**单评委的分数不可信，评委之间的分布才可信。**

1.4 明确的边界（不做什么）

不做	理由
PPTX 解析	需要 officeparser/JSZip 全套 + 大量版式噪声，而本作品语料是论文/文档/简历，PDF+DOCX 已覆盖
知识图谱 / 3D / 拖拽画布	与”练答辩”这个目标无关，是演示性功能
替用户改材料	产品是”预演”，不是”代写”；改材料是用户的动作，产品只负责审计
给材料打总分并声称”你的论文 60 分”	见 §2.4：材料风险分由服务端聚合，不由模型拍脑袋

---

2. 意图控制策略与可证伪生成

> 官方措辞是”意图控制策略（你是如何设计上下文和 Prompt 的）“。本章标题加上”可证伪生成”，因为本作品在意图控制之上多了一层：**每条模型输出都必须带一个可被机器推翻的锚点。**这是全篇最核心的设计。

2.1 五阶段 Prompt 链与各自的意图

模型被调用五次，每次只做一件事。**拆分的目的不是复用，是让每一环的失败都可定位。**

阶段	输入	输出	意图
`claims`	材料切块（每批 5 块）	质疑点清单	找出材料里”一问就露怯”的地方
`personas`	材料 + 质疑点清单	3 位评委及各自首问	把质疑点分配成三个不同视角
`question`	评委 + 目标质疑点	一句提问	把质疑点写成答辩现场会问出口的话
`evaluate`	问题 + 评委 + 材料原文 + 考生回答	分数 + 缺口 + 锚点 + 是否追问	判这一次回答站不站得住
`report`	全部问答 + 服务端聚合的风险分	两栏报告 + 高风险清单 + 建议	汇总成本场可读的风险结构

为什么不是一个大 Prompt 一次出全部？一个大 Prompt 会让”报告质量差”归因到”模型今天状态不好”；拆成五环后，报告差能追到”claims 抽得不对”或”evaluate 判分恒定”这类可单独修的问题。这一点在 §2.5 的对比评测里被直接验证。

2.2 上下文设计的四条硬规则

上下文里塞什么、塞多少，直接决定模型会不会编。四条规则都是踩过坑之后写死的：

规则 1：块号由服务端给，模型只许引用，不许自己数

```
[3] (P2 方法) 我们采集了某市三所小学一学期的课后服务记录，共计 1240 条……
[4] (P2 方法) 实验在同一批数据上进行，未划分训练集与测试集。
```

`lib/prompts/claims.v1.ts:buildClaimsUser` 里 `byIdx = new Map(blocks.map(b => [b.idx, b]))`，**第二个参数是块的编号，不是数组下标**（早期版本按 `blocks[idx]` 按位置取，过滤短块后两侧必然不等，导致模型凭空编 quote）。本批没有可用块时，直接提示模型返回空清单，而不是让它自由发挥：

```
（本批没有可用片段，请直接返回 {"claims": []}，不要编造内容。）
```

规则 2：quote 必须逐字 8–60 字，这是机检唯一的前提

模型写"引用"时天然会润色、缩写、跨段拼接。所以我们规定：

- 逐字，**不许改写、不许补字、不许加省略号、不许跨块拼接**；
- 长度 8–60 字（太短无法定位、太长容易跨块）；
- 长度与"逐字"两条同时进 zod schema 和 `lib/locator.ts` 机检。

规则 3：给 `absence` 一条合法出口

这是 v1 最关键的改动，也是 §2.5 里差异最大的地方。

如果只允许 `quote`，模型遇到"材料里确实没有这个东西"时，唯一能输出的写法就是**编一句看起来像原文的话**——而编出来的引用会被机检判 0，直接污染命中率。所以我们显式开一条出口：

```
{ "kind": "absence", "block_idx": 7, "quote": "", "scope": " 第 2 章数据与方法" }
```

`absence` 表示"原文里没有可直接引用的句子",用 `scope`(≤60 字)说明**查过哪儿。**它走 `lib/locator.ts:verifyAbsence` 的 scope 合法性校验，**不进 quote 命中率的分母**——因为对"缺失类"结论算逐字命中率没有意义，算进去只会让指标失真。

规则 4：页码与章节由服务端回填

模型不输出 `page` / `section`，只回 `block_idx`；页码和章节从块表里查出来填。理由：模型很擅长给出"看起来合理的页码"（P3、P7），而那些页码是编的。**凡是模型能编的东西，都不让它输出；凡是能查的，都去查。**

2.3 强制打分为什么会被打歪——以及怎么治

`evaluate` 阶段要模型给 0–5 分。直接说"请给这次回答打分"，结果一定是**恒 3 分**：不是答得刚好三分，而是模型在回避极端判断。

治法是**把每一档写死成可判定的条件**（`lib/prompts/evaluate.v1.ts:SCORE_LABEL`）：

分	判定条件（写进 Prompt）
5	直接给了确切答案，有具体数字 / 方案 / 边界
4	答案成立，只差一处细节
3	方向对，但关键处含糊
2	答非所问或分明显
1	基本没接上
0	空答，或与材料自相矛盾

再叠加两条顺序约束：

1. **先写 gaps（缺口），再定 score。**顺序不许颠倒——反过来就是"先有分数，再补理由"，理由会为分数服务。
2. **locators 至少 1 条。**说"你这里含糊"，必须指到原文哪儿，或者明说查过哪儿但没有（`absence`）。

以及一条防"为多问而多问"的规则：

> `followup.resolved`：已经答清的写 `true`，**不要为了多问一轮而把已经答清的说成 false**；已追问 2 次的一律给 `null`。

追问收敛不交给模型自觉，见 §2.6。

2.4 材料风险分为什么不由模型给

这是全篇最重要的一条产品决策，也是 C13 的落点。

如果让模型一边写评语一边给材料风险分，会出现两头失真：

1. **强制打分会让模型编引用。**为了给 81 分找个理由，它会凑出"依据原文：……"里根本不存在的一句；
2. **同一份材料每场分数不一样。**材料风险是**材料属性**，不该因为今天考生答得好就降到 70。

所以：

量	谁出	极性与口径	能不能比
`risk_score`（材料风险）	**服务端从 claims 聚合**，模型只写评语	越大越糟，0–100	同一份材料**恒定**，可跨场比较
`performance.score`（临场表现）	模型出分，0–5 经 `scoreToPercent` 折算 0–100	越大越好	只反映这一场答得怎样

两条硬约束写进代码与 schema：

- **不许共名**：一个叫 `risk_score`，一个叫 `performance.score`；
- **不许共颜色映射**：报告页两栏用不同色系，视觉上不会被误读成同一维度。

报告 Prompt 里明写：

> 【材料风险（服务端已聚合，仅供参考，**不许你输出分数**）】……`material_risk` 每条只有 `dimension` / `comment` / `locators`，**没有 score 字段。**

并且 `lib/prompts/report.v1.ts` 的 schema 里确实没有 `score`——模型就算想给也给不出来（会被 zod strip）。§2.5 的对比评测里，拆掉这条约束后的 v1 **2/3 轮越界给了材料分。**

2.5 Prompt 演进证据：v1 vs v2 同输入对比

> 完整产物在 `docs/prompt-eval/`：`diff.md`（聚合表 + 原始 JSON + 结论）+ 每组 `v1.json` / `v2.json`（各 3 轮全量）+ `v1.system.txt` / `v2.system.txt`（两套 system 原文留档）。

方法说明（重要）：v1 不是某个历史版本，而是把 v2 的每条硬约束**逐条拆掉**之后做的对照物。这样做的理由是：历史版本会同时改十件事，看不出是哪条规则在起作用；逐条拆解能把"拆掉哪条、哪条就出问题"对上号。三轮/版，用**通过率**而不是单次结果下结论（见 §2.5.4 的教训）。

2.5.1 `claims` 组

指标	v1（拆掉硬约束）	v2（现行）
zod 结构通过	2 / 3 轮	3 / 3 轮
用 `absence` 出口	**0 / 3 轮**	**3 / 3 轮**
quote 逐字命中	18 / 18	18 / 18

v1 的失败形态：`claims.6.type: Invalid option: expected one of "method"|"engineering"|"value"`——把 `dimension` 的取值（`expression`）写进了 `type` 字段。以及最关键的：**拆掉 absence 出口后，v1 三轮一次都没用过它**，全部试图给 quote。

2.5.2 `evaluate` 组

指标	v1	v2
zod 结构通过	**0 / 3 轮**	3 / 3 轮
分制越界（改成百分制）	**3 / 3 轮**（score=18）	0 / 3 轮

v1 把 `score: 0-5` 改成开放式描述后，模型三轮全部自作主张改用百分制。报错原文：

```
score: Too big: expected number to be <=5
locators.1.quote: Too big: expected string to have <=60 characters
```

这条差异最能说明"把判定条件写死"的必要性：**不给范围，模型就会自己发明范围。**

2.5.3 `report` 组

指标	v1	v2
quote 逐字命中	**0 / 19**	**9 / 9**
locators 写成字符串数组	**3 / 3 轮**	0 / 3 轮
越界给材料分	**2 / 3 轮**	0 / 3 轮
high_risk 的 claim_id 可溯源	通过	通过

v1 的 locators 写成 `["第 2 章提到……"]` 这种字符串数组，而 v2 要求对象数组（`{kind, block_idx, quote}`）。这个形状差异会让**整份报告无法进入机检**，是最贵的一条。`claims` 组的 `high_risk` claim_id 两组都没编造——说明"只许从清单原样复制"这条约束比较容易被满足，不是瓶颈。

2.5.4 一条方法论教训：单次对比会被随机性骗

`claims` 组第一次跑，v1 违规 0 条，看起来"约束没什么用"；第二次跑，v1 直接把 `dimension` 写成 `expression`。**结论会来回翻。**所以我们改成每组每版跑 3 轮取通过率，并把这条写进了 `diff.md` 的备注。

这不是统计学意义上的显著性检验（样本太小），但它足以避免"看一次结果就下结论"这种更严重的错误。

2.6 追问收敛：纯函数，不交给模型自觉

Prompt 里写了"已追问 2 次就不要再追"，但**规则不能只写在 Prompt 里**——模型会忘、会被上下文带偏、会为了"显得认真"多问一轮。

收敛规则写在 `lib/state.ts` 的纯函数里，模型只提供素材（`followup` 建议），最终是否追问由服务端判：

- 同一质疑点已追问 2 次 → 一律不追；
- 模型给 `resolved: true` → 不追；
- 本场主问已满（quick 档 3 位评委 × 1 主问）→ 进报告；
- 预算耗尽 / 用户点"结束并出报告" → 立即出报告。

纯函数的好处：**可单测**（`tests/state.test.ts` / `tests/state-stage4.test.ts` 共 45 条），而且行为与模型当天状态无关——这是"追问收敛率 100%"（§5）能成立的前提。

2.7 五阶段之外的守门：三道校验

每次模型返回都要过三道闸，任一道不过就带错误信息重试（最多 2 次）：

1. `lib/schemas.ts` **的 zod 校验** ——结构、枚举、长度；
2. `lib/locator.ts` **的机检** ——`quote` 走逐字命中，`absence` 走 scope 合法性；
3. **归属校验** ——`followup.target_claim_id` 必须在本场的候选清单里，否则**当作不追问处理**（而不是报错中断）。

第 3 条的设计意图：模型偶尔会”幻觉出一个 claim_id”，这时最坏的结果应该是”少问一轮”，而不是”整场会话崩掉”。

---

3. 架构设计

> 官方措辞是”架构设计（AI 生成的代码采用了什么架构，你如何约束它）“。两个问号：架构本身是什么；**怎么约束 AI 不乱写。**

3.1 技术栈与选型理由

层	选型	版本	为什么是这个
框架	Next.js（App Router）	**15.5.28**（锁版，不升 16）	路由 + API 同仓，SSE 用 Route Handler 直接返回 `ReadableStream`，不需要额外服务
语言	TypeScript	5.x	模型输出必须过类型化 schema，`any` 会让机检失效
样式	Tailwind v4	4.x	设计 token 集中管理
存储	`node:sqlite`	Node 24 内置	零原生编译——评委机器上 `npm install` 不会卡在 better-sqlite3 的编译链上（可复现性的第一道保障）
校验	zod	**4.6.7**	同上，schema 即契约
PDF	`pdfjs-dist`	**5.4.149**	逐页取文本 + 页码，锚点模型需要页码
DOCX	`mammoth`	1.13.0	段落结构保留好，足够切块
模型	DeepSeek（OpenAI 兼容接口）	`deepseek-chat`	运行时用选手自己的 key

依赖锁版 + `--omit=optional` + `scripts/polyfill-canvas.mjs` 这三个动作合起来解决同一件事：让 `npm install` 在任何一台干净的机器上不编译原生模块、不因为 canvas 缺失而崩。

3.2 分层与依赖方向

```
app/                          页面与 API 路由（只做 HTTP 边界：解析、校验入参、返回）
  api/**/route.ts               请求编排 + 错误码
  **/page.tsx                   展示与交互（client 组件，不含业务规则）
components/                   可复用 UI（无业务逻辑）
lib/                          全部业务逻辑（纯函数优先，可直接单测）
  parse.ts  extract.ts          材料 → 块 → 质疑点
  schemas.ts                    所有 LLM 输入输出与 API 载荷的 zod 契约
  prompts/*.v1.ts               五个阶段的 system + user 构造（纯函数）
  llm.ts                        唯一模型出口（MOCK 回放 / 真实调用）
  locator.ts                    锚点与逐字机检
  state.ts                      追问收敛状态机（纯函数）
  grade.ts report.ts            判分聚合与报告组装
  db.ts                         唯一 DB 入口（懒连接 + 事务）
```

三条硬规则：

1. `lib/` **不 import** `app/`（依赖单向，否则业务逻辑与 HTTP 耦合，无法单测）；
2. **模型调用只能从** `lib/llm.ts` **出去**（一处出口 = 一处超时/预算/重试/回放逻辑）；
3. **DB 只能从** `lib/db.ts` **进出**（一处 = 一处建表与事务）。

3.3 约束 AI 生成代码的六条机制

这一节直接回答官方的第二个问号。**约束不是靠"跟 AI 说别乱写"，是靠让乱写的代码跑不起来。**

#	机制	具体做法	拦住什么
1	**zod 契约先行**	先写 `lib/schemas.ts`，模型输出、API 载荷、fixture 全用同一套 schema	字段名漂移、枚举值乱写、模型返回旧结构
2	**纯函数隔离**	状态机（`state.ts`）、机检（`locator.ts`）、聚合（`grade.ts`）都是纯函数	规则散落在 route handler 里，改一处漏三处
3	**测试闸门**	414 条单测 + `npm run test:fixtures`	改 prompt/schema 后 mock 静默返回旧结构
4	**安全基线 S1–S6**	白名单 + 魔数 + 服务端落盘名（见 §3.5）	上传通道被打穿
5	**禁用** `dangerouslySetInnerHTML`	模型输出一律文本切分渲染，quote 高亮用字符区间	XSS ——模型输出是不可信输入
6	**铁律清单 + AI 日志**	`CLAUDE.md` 九条铁律 + 每次会话写 `docs/ai-logs/`（含 Bug 与修复过程）	AI 反复踩同一个坑

第 6 条最容易被低估：`docs/ai-logs/` 不是给评委看的装饰，是给下一个会话的 AI 看的。里面记的都是"这个项目上 AI 具体怎么错的"，例如：

- 把 `max-width: 56ch` 误判成 400px 横向溢出的根因（`max-width` 根本不可能撑宽父容器）；
- 驱动脚本里 `token` / `meta` / `evaluation` 三处字段名全猜错，全静默 `undefined`，记录全是空表；
- `z.object` 会静默 strip 未知字段，用 `safeParse` 结果判"模型有没有越界给材料分"永远为 false。

3.4 锚点模型：全篇最核心的数据结构

```ts
type Locator = {
  blockIdx: number;       // 材料切块后的块号（服务端给，模型只许引用）
  page?: number;          // PDF 才有，服务端回填
  section?: string;       // 章节路径，服务端回填
  quote: string;          // quote 类：逐字 8-60 字；absence 类：空串
  kind: 'quote' | 'absence';
  scope?: string;         // absence 类：查过哪儿，≤60 字
};
```

这不是"引用页码"，是"块 + 逐字文本"的双重定位。纯页码的问题是模型会编页码；块号由服务端分配所以编不了，而逐字文本又必须与原文真正对得上，两把锁同时锁住。

逐字机检怎么做（`lib/locator.ts`）

四级回落，每一级的命中半径都被刻意限制：

级别	做法	命中半径
1. 原地命中	在指定块里 includes	0
2. 邻域命中	在 ±2 块范围内找	±2
3. 全篇兜底	全材料找（记录移到哪一块）	全篇
4. 未命中	判 verified = 0，**留在分母里**	—

第 3 级”全篇兜底”存在的原因是模型数块号会差几个；但第 4 级**必须留在分母**——否则”命中率”就退化成了”命中的里面有多少命中”，这是自欺欺人的指标。

absence 走另一条路（verifyAbsence）：去掉空白后双向包含 + 区间正则，判定 scope 是否落在已知章节里，reason 只有两种结果：’scope 合法’/’scope 未落在已知章节’。**命中率分子只认**’scope 合法’。

3.5 安全基线 S1–S6

编号	要求	实现
S1	上传白名单 + 体积上限	三道闸：扩展名 → 声明 MIME → **文件头魔数**（lib/security.ts，20MB 上限）
S2	落盘名不用用户输入	落盘名只由服务端 randomUUID 生成；用户文件名只能进 DB 当展示名，**绝不参与路径**
S3	模型输出不可信	一律过 zod，渲染一律文本切分，禁 dangerouslySetInnerHTML
S4	会话隔离	每个 session 的 turns / personas / claims 都按 session_id 过滤
S5	错误不泄漏内部信息	API 返回统一错误码（E_NO_SESSION / E_TURN_MISMATCH / …），不抛栈
S6	预算上限	每场会话最多 30 次模型调用（MAX_CALLS_PER_SESSION），单次 30s 超时

S1 的三道闸里**只有魔数是骗不过去的**：扩展名和 MIME 都是上传方自己写的。所以魔数不一致时一律拒绝（把.exe 改名.pdf 会被挡），不做”以魔数为准就放行”的宽容处理。

3.6 SSE 流式与思考态

官方 15% 原文点名”如流式、思考可视化”，所以这是不可砍项。

实现：app/api/sessions/[id]/stream/route.ts 返回 ReadableStream，逐 token 推送（TOKEN_FLUSH_CHARS 分批），事件类型 token / meta / done / error。

- meta 事件在首字前发出，带评委信息 → 前端据此渲染”谁在问”；
- done 事件带 turnId → 前端用它提交回答，**不靠猜**；
- MOCK 模式下也走同一条 SSE 路径（回放），所以前端只有一套代码。

3.7 MOCK 回放：架构上的一等公民

MOCK_LLM=1 时，lib/llm.ts 不发起真实请求，而是按”序号驱动”从 fixtures/llm/*.json 回放。

三个设计细节，每个都踩过坑：

1. **游标按** `${sessionKey}:${phase}` **分键。**早期只按 `sessionKey` 计数，而 question/evaluate/report 三个阶段共用同一个 `session.id`，于是每次 `/answer` 里的 evaluate 会把 question 游标往前推一格 → 三问变成 q1/q3/q3。
2. **游标挂在** `globalThis.__mockCursor`。 Next 热重载会重新求值 `lib/llm.ts`，模块顶层的 `const mockCursor = new Map()` 被重建、游标归零 → 多次提问会取回同一问（实测 q1/q1/q2）。
3. `resetMockCursor(sessionKey)` **按前缀清理该场所有 phase。**

这三条合起来才让" 零 key 演示" 真的走得通。**它们不是测试技巧，是产品功能**——评委机器上没有 key，MOCK 回放是唯一能让他看到完整闭环的路径。

---

4. 可复现性论证

> 官方 25% 明确写" 评委可一键复现"。本章给三层证据：代码层（一个命令）、数据层（fixture 与种子）、行为层（同输入同输出）。

4.1 三层可复现，从强到弱

层	主张	手段	强度
L1 无需 key 的完整闭环	任何人 `npm install && npm run demo` 就能走完上传 → 质询 → 报告 → 复练	MOCK 回放 + 内置样例播种	**最强**：不依赖网络、不依赖 key、不依赖模型当天状态
L2 真实模型的管道可复现	有 key 时，同一材料的材料风险分恒定、锚点可复检	`risk_score` 服务端聚合 + 双重采样实测（§5.2）	中：临场表现分有波动 (模型出分)，但材料侧恒定
L3 输出结构可复现	同一 prompt 版本的输出结构一致	zod schema + `test:fixtures` + prompt 版本号	强：结构是契约，不是概率

诚实的边界：L2 里 `performance.score` 两场不完全一致 (实测 {30, 25, 30, null} vs {25, 30, 20, null})。这是模型的固有随机性，我们不声称能消除它——我们能声称的是**材料风险分不受它影响**（§5.2）。

4.2 一个命令

```
npm install
npm run demo          # MOCK_LLM=1 + 自动播种内置样例 + 起服务
```

README 自检小节写明了三条命令与期望结果：

命令	期望
`npm test`	414 个单测全绿
`npm run test:fixtures`	39 项检查·0 失败（三层：zod / 机检 / 锚点去重）
`npm run demo`	零 key 走完上传 → 报告 → 复练

4.3 fresh clone 为什么曾经走不通（一个真实的可复现性故障）

这是本项目最值得写进文档的一个 Bug，因为它**只在" 干净的机器" 上出现**：

1. MOCK 模式下前端禁用上传（否则用户会传一份没有回放剧本的材料，评委将无法据此提问）；
2. 而空库里 `/api/demo` 返回 `E_NO_DEMO_DOC` → 用户点" 用内置样例开始" 也失败；

3. 更隐蔽的是：personas fixture 锚定**具体 claim_id**，如果播种时重新生成 UUID，录制好的 opening 就一个都匹配不上 → 三位评委被过滤成 0 位。

修复：scripts/dump-demo-seed.mjs 导出 seed-demo-data.json，**保真 claim_id / block_idx / quote / dimension / risk**；scripts/demo.mjs 起服务前 seedIfMissing() 幂等灌库（只在库里没有"有 claims 的 sample-paper.docx"时才写，不会覆盖真实模式的数据）。

这个 Bug 的教训：可复现性不能靠"我这台机器上能跑"。我们现在的验证方式是 scripts/verify-demo-loop.mjs ——一个脚本从零驱动完整闭环（GET /api/demo → 建会话 → 三问 → 报告），输出 [OK] 零 key 闭环走通，共 3 问，评委 3 位，报告生成成功。**只有这条跑通才算可复现。**

4.4 固定随机性的手段（以及为什么不够）

手段	状态	说明
temperature 固定	已完成 已做	默认 0.2，写进 lib/llm.ts
response_format: json_object	已完成 已做	结构约束，不是采样约束
Prompt 版本号	已完成 已做	CLAIMS_V1 / EVALUATE_V1 / REPORT_V1 常量
risk_score 服务端聚合	已完成 已做	让材料侧分数完全脱离模型采样
seed 参数	未完成 未做	DeepSeek 接口未暴露 seed；这是已知局限，不假装解决
双采样一致性实测	已完成 已做	用实测数字交代波动幅度，而不是声称"无波动"

不做的诚实：我们不声称输出逐字可重复。我们声称的是"**结构可重复、材料侧分数可重复、锚点命中可复检**"，并且用 §5 的数字量化了模型侧的波动。

4.5 锚点可复检

任何一条结论都能点回原文。这不只是 UI 功能，也是可复现性的一环：**评委不必相信报告，他可以自己检查。**

报告页每条结论都带 BLOCK n 锚点，点开在左侧原文栏高亮命中位置；quote 与 absence 两类有不同视觉(前者是原文有的句子，后者是原文没有的，说明该补什么)。

4.6 模型版本漂移的风险

deepseek-chat 是别名，指向的模型可能更新。我们的处理：

- 报告中不写"模型版本"，避免给出无法验证的精确声明；
- **材料侧分数不受影响**（服务端聚合）；
- 结构由 zod 保证，模型升级导致的格式漂移会被 schema 拦下并重试；
- 若模型行为显著变化，重录 fixture（scripts/record-fixtures.mjs）即可，这是一条已踩过且写进文档的操作路径（§2 改 prompt 后我们就重录过一次）。

---

5. 评测与数据

官方要求"评测数字标注样本量"。本章每个数字都带样本量，并对**哪些数字可以宣传、哪些不能**做了明确区分。

5.1 八项机检指标（scripts/eval.mjs 自动产出）

评测方法：用真实 key 驱动 2 场完整 quick 会话（同一份内置材料），从报告与 turns 表机检。产物 docs/eval/eval-result.json。

#	指标	口径	样本量	结果	目标	判定
1	引用命中率	verified=1 的 quote 类 **/ quote 类总数（含未命中）**	11	**100.0%**	≥95%	已完成
2	缺失类检索覆盖率	scope 合法 / absence 类总数	1	**100.0%**	≥95%	已完成
3	类型抽检一致率	人工逐条判 model_type 与 text 是否匹配	12	**100%** (12/12)	≥80%	已完成
4	双采样一致性	同答案两次评测 |Δscore|≤1 的占比	3 对	**100%**	≥75%	已完成
5	追问收敛率	≤2 轮内结束的占比	2 场	**100%**	≥90%	已完成
6	证伪有效性	人工判 quote 是否真支撑该质疑	12	**100%** (12/12)	≥80%	已完成
7	quick 档时长	上传 → 报告可读（秒）	2 场	**最长 43.1s**	≤480s	已完成
8	成本	每场 token 与首字延迟	2 场	记录于 llm_logs 表	记录即达	已完成

两条口径必须说清楚（否则数字会误导）：

1. **命中率的分子包含"邻域命中"（±2 块）**，不是只算原地命中。这是有意的：模型数块号差 1–2 个是已知且可接受的行为，真正要防的是"编一句原文里没有的话"。但**未命中的条目留在分母里**，不做剔除。
2. risk_score **不进这张表**，因为它由服务端从 claims 聚合，两场恒等（实测 {logic 81, evidence 80, depth 50, expression null}）。一个"恒定"的值不是"稳定性指标"，它是结构性事实。能衡量稳定性的只有 performance。

5.2 双采样实测：哪些值恒定、哪些会变

量	第一场	第二场	解读
材料风险 risk_score	{81, 80, 50, null}	{81, 80, 50, null}	**恒等**——同 claims 聚合，与模型采样无关
临场表现 performance.score	{30, 25, 30, null}	{25, 30, 20, null}	**有波动**——模型出分，这是它的固有随机性
维度覆盖	3/4 维	3/4 维	稳定

这张表是本章最重要的一页：它说明我们**没有**假装模型输出稳定，而是把稳定与不稳定的部分分开了——材料侧（可宣传的部分）恒定，模型侧（有波动的部分）如实交代。

5.3 三份真实使用记录

零 key 回放证明”管道通”，但不能证明”对真实材料有用”。所以另跑三份真实材料（真实 key）：

材料	体量	抽出 claims	quote 命中	亮点
合成论文（城市社区废旧纺织品回收行为研究）	15 blocks / 961 字	**21 条**（evidence 11 / logic 7 / depth 3）	**18/19**	质疑点集中在”意愿当行为” “300 米无出处”“CFI=0.93 低于 0.95 却称达标”
一页薄简历	6 blocks / 270 字	**5 条**（evidence 3 / depth 1 / logic 1）	**5/5**	材料薄时布局不塌 (高风险 5 条而非 8 条)
本项目说明书（狗粮自查）	476 blocks / 35550 字	**71 条**（evidence 21 / logic 17 / depth 30 / expression 3）	**56/67**	评委在本项目自己的文档里指出了 8 条真实缺口

合计 quote 命中 **79/91 = 86.8%**。三份记录全文在 `docs/usage/`，每份含「它问了什么 / 它说的缺口（原文）/ 报告要点 / 看完之后我改了什么」。

狗粮自查发现了什么（这是最诚实的一节）

第 3 份材料是本项目自己的技术文档。三位 AI 评委指出的 8 条缺口里，有几条是真的：

1. 可复现性主张依赖 LLM，但没交代固定输出手段（温度/种子/结构化约束）；
2. 自查本身没有”跑了几遍、两遍一致吗”的记录；
3. polyfill 只验证了 1 份 PDF；
4. 懒加载只把抢锁时机推后，没有压测数据；
5. `y 容差 2.5` 没有标定过程；
6. 5 分制没有锚点、评委间一致性未验证；
7. 声称”未复用既有代码”但无举证；
8. 三位 AI 评委的质询质量本身无对照评估。

其中 4 条已经改进（口径补进本文档、数字写进 progress、polyfill 表述收窄到”该份单栏 PDF”、懒加载从”根因已解决”降级为”规避启动期抢锁”），**4 条如实挂账**（y 容差标定、5 分制锚点、评委质量无对照、重复执行记录）。

为什么要把这段写进技术文档：官方要求”可复现性论证”。一件说得出自己哪里不能复现的作品，比一件声称全都可复现的作品更可信。

5.4 报告的”评分散点”：为什么按评委聚合

报告右栏每一维都画评委打分散点。第一版按”每一次提问”画点，实测发现一个问题：

> 某一维是同一位评委（陈评委）问两次，各给 1 分。逐问画点会显示成两个点，看起来像”两位评委各给 1 分”——正好糊掉了要看的东西。

改为**按评委聚合**（一位评委取均值一个点），`dots.length` 才是真正的评委数，并额外画一条竖线标”本维汇总分”**。竖线与散点位置之差就是分歧幅度。**

实测示例：`evidence` 维汇总 30%，陈评委两问均值 1.0 → 20%，竖线和散点差 10 个百分点——这个差距告诉用户”这位评委比汇总判断更悲观”。

5.5 这些数字不能用来声称什么

明确写在这里，防止被过度解读：

- 未完成 **不能说”命中率已稳定在 100%“**——样本只有 11 条与 1 条 absence，超长文档实测是 83.6%；
- 未完成 **不能说”输出逐字可重复”**——`performance.score` 两场不同；
- 未完成 **不能说”评委质量经过评估”**——没有对照组（见 §6）；
- 已完成 **可以说”管道通、机检生效、材料侧分数可复算、每条结论可回原文”**——这些都有脚本与数据支撑。

---

6. 局限与下一步

6.1 已知局限（按严重程度排序）

#	局限	影响	现状
1	**三位 AI 评委的质询质量没有对照评估**	无法证明”AI 评委比随便问更好”	只有三份使用记录的定性观察；定量对照需要人工标注基线，成本超出预算
2	**absence 类样本极少（1 条）**	缺失类覆盖率 100% 建立在一个样本上，统计意义弱	不为了凑数字造假材料；如实标注样本量
3	**超长文档 quote 命中率下降（83.6%，56/67）**	476 块的材料上模型数块号误差超出 ±2 邻域	已知；可选改进是全篇兜底，但会削弱”块号必须引用”的约束
4	**模型输出不能逐字复现**	`performance.score` 场间波动	用双采样交代波动幅度，不假装消除
5	**5 分制锚点未写进给评委看的文档**	评分标准对用户不完全透明	挂账（11-11 补）
6	**PDF 覆盖不完整**	扫描件、双栏、复杂表格不支持	在 README「已知不支持」中写明
7	**单机 sqlite，不考虑并发写**	多人同时使用会锁	演示场景是单机，如实写明不涉及
8	`y` 容差 `2.5` **未标定**	页码定位的容差是经验值	挂账

6.2 下一步（如果继续做）

按”对评分影响 / 成本”排序：

1. **评委质量对照评估** ——同一材料分别用”普通问答”与”三评委质询”跑，人工判哪边问得更深。这是最能补强 §6.1 第 1 条的一步；
2. **5 分制锚点抄进报告页** ——成本极低，直接提升体验分；
3. **超长文档块号容差** ——对超长材料放宽邻域或引入二次定位；
4. **PDF 多栏/表格** ——需要版面分析，成本较高，仅在语料需要时做。

6.3 不打算做的

- 替用户改写材料（产品定位是审计，不是代写）；
- 多人协作 / 云端部署（省赛演示是单机，做这些只会稀释核心闭环）；
- 任何与”练答辩”无关的演示性功能（知识图谱、3D、拖拽画布）。

附录 A：AI 协同记录索引

每次 AI 会话都有 docs/ai-logs/YYYYMMDD-主题.md，六段结构：目标 / Prompt 原文 / AI 输出摘要 / 我的修改 / **Bug 与修复过程** / 遗留问题 + commit。

值得单独提的几条（都写进了 docs/ai-logs/）：

主题	AI 具体怎么错的	修复
mock 回放串问	游标单 key + 热重载归零，双根因叠加	键改 sessionKey:phase + 挂 globalThis
fresh clone 走不通	空库无样例 + claim_id 对不上	保真导出种子 + 幂等播种
评测命中率恒 0	verifyQuote 参数签名写错，把"全命中"报成"全未命中"	改三参调用
越界判据永远 false	z.object 静默 strip 未知字段	改读模型原始输出
400px 横向溢出	误判 max-width 为根因（它不可能撑宽父容器）	iframe 探针定位到单行 flex

这五条的共同点：AI 的错误都不是"写不出代码"，而是**写出了能跑但结论错的代码**（静默失败）。所以本项目的测试策略重点是"用机检证明结论"，而不是"用 lint 证明语法"。

附录 B：目录与文件索引

路径	内容
lib/	16 个模块，约 3300 行业务逻辑（纯函数优先）
app/api/	11 个 Route Handler
tests/	28 个测试文件，414 条单测
fixtures/llm/	五阶段回放剧本（{notes, seq:[{inputHash, response}]}）
fixtures/demo/	内置样例（sample-paper.docx + blocks.json）
scripts/	录制 / 校验 / 评测 / 复现验证脚本
docs/eval/	三份评测材料 + 评测结果 JSON
docs/prompt-eval/	三组 v1 vs v2 对比（含原始 JSON 与两套 system）
docs/usage/	三份真实使用记录
docs/ai-logs/	每次 AI 会话的过程记录（含 Bug 与修复）